AgileFlow AI

File 07 - Task Assignment and Branching Rules

Document Purpose

- Provide a clear task assignment and branch management plan for the AgileFlow AI implementation team.
- Prevent duplicate work, missing functions, broken merges, and AI-agent generated conflicts.
- Define who owns each component, which files each member can edit, and how pull requests must be reviewed.

1. Project Team and Component Ownership

Member	Student ID	Owned Component	Primary Branch
N A Illangasinghe	IT23391536	Requirement Gathering and Analysis Agent	feature/agent-requirements
Kulunu Kasthuri Arachchi	IT23375628	Project Management and Design Agent	feature/agent-design-planning
W M D D Karunarathna	IT23145320	Implementation and Git Automation Agent	feature/agent-implementation
R A K Hansika	IT23140998	Testing, Validation and Security Agent	feature/agent-testing-validation
All members	-	Central Orchestrator, Shared Knowledge Base, Traceability, Final Integration	feature/orchestrator / feature/reports-traceability

2. Recommended Starting Split for First Implementation Task

Sub-group	Members	Main Responsibility	Deliverables
Group A - UI/Figma Design Team	2 members	Design the web application UI, screen flow, dark purple theme, reusable components, and Figma prototype.	Figma pages, component library, page flow, UI screenshots, UI handoff notes
Group B - Project Foundation Team	Group leader + 1 member	Initialize the repository, folder structure, documentation, frontend, backend, database connection, and core workflow models.	Working base repo, docs, API contracts, shared schemas, branch rules, initial project APIs

- The UI team should start in Figma first and should not create a separate frontend structure without approval.
- The foundation team creates the real project structure and gives UI team the approved folders and naming rules.
- After the foundation is merged into develop, each member starts coding inside their assigned component folder.

3. Branching Strategy

```
main
  stable final version only


develop
  integration branch for completed and reviewed features


feature/project-foundation
  initial repo, frontend/backend setup, database, docs, base workflow


feature/ui-design-system
  UI layout, design system, reusable frontend components


feature/agent-requirements
  Requirement Gathering and Analysis Agent


feature/agent-design-planning
  Project Management and Design Agent


feature/agent-implementation
  Implementation and Git Automation Agent


feature/agent-testing-validation
  Testing, Validation and Security Agent


feature/orchestrator
  workflow control, approval gates, routing, retry loop logic
```

```
feature/reports-traceability

  final reports and traceability matrix
```

- No one should directly push to main.
- No one should directly push to develop after initial setup; use pull requests.
- Each member must work on their assigned feature branch.
- Only the group leader or assigned reviewer should approve merges into develop.

4. Folder Ownership Rules

Area	Folder	Owner / Allowed Editor
Backend core	backend/app/core, backend/app/database, backend/app/main.py	Group leader / foundation team only
Frontend layout	frontend/src/layouts, frontend/src/components/common, frontend/src/routes	Group leader + UI design team
Requirement Agent	backend/app/modules/requirements, frontend/src/pages/requirements	Requirement Agent owner
Design Agent	backend/app/modules/design, frontend/src/pages/design	Design Agent owner
Implementation Agent	backend/app/modules/implementation, frontend/src/pages/implementation	Implementation Agent owner
Testing Agent	backend/app/modules/testing, frontend/src/pages/testing	Testing Agent owner
Orchestrator	backend/app/modules/orchestrator, frontend/src/pages/orchestrator	Group leader + integration reviewer
Reports / Traceability	backend/app/modules/reports, frontend/src/pages/reports	Shared, but changes must be reviewed
Documentation	docs/	All members can propose changes; group leader approves shared contracts

5. Protected Files

Do not edit these files without permission

• backend/app/main.py
• backend/app/core/config.py
• backend/app/database/connection.py
• backend/app/models/common.py
• frontend/src/App.jsx or App.tsx
• frontend/src/routes/index.jsx or router file

- frontend/src/layouts/MainLayout.jsx
- docs/API_CONTRACT.md
- docs/DATA_SCHEMAS.md
- docs/AI_AGENT_RULES.md

6. Member-wise Development Tasks

Member / Component	Main Tasks	Main Output	Should Not Modify
Requirement Gathering and Analysis Agent	Requirement input UI; client input processing; requirement JSON; SRS summary; user stories; acceptance criteria; ambiguity/missing-info report; H1 approval.	Approved requirement package with REQ IDs.	Design generation, implementation logic, final validation logic.
Project Management and Design Agent	Backlog generation; sprint plan; API plan; database schema; UI screen/wireframe plan; requirement-to-design mapping; H2 approval.	Approved planning and design package.	Requirement extraction logic, code generation logic, test execution logic.
Implementation and Git Automation Agent	Implementation dashboard; project structure generation; backend/frontend prototype modules; Git branch/commit simulation; logs; limited self-fix; H4 approval.	Generated prototype foundation with code metadata and logs.	Requirement extraction, design generation, final validation structure.
Testing, Validation and Security Agent	Pre-code test generation; H3 approval; test execution dashboard; bug reports; security findings; H5 approval; final validation report.	Approved test package, validation report, bug report, security report.	Implementation generation logic, design backlog logic, requirement processing logic.

7. Implementation Order

1. Initialize GitHub repository and create main/develop branches.
2. Create documentation files: README, CONTRIBUTING, API_CONTRACT, DATA_SCHEMAS, AI_AGENT_RULES, TASK_ASSIGNMENT.
3. Create frontend and backend base applications.
4. Create common models: Project, Approval, WorkflowEvent, TraceabilityLink.
5. Create Orchestrator basic workflow state and next-step logic.
6. Build Project Dashboard, Create Project page, and Project Workflow page.
7. Implement Requirement Gathering and Analysis Agent with H1 approval.
8. Implement Project Management and Design Agent with H2 approval.
9. Implement Testing Agent pre-code test planning with H3 approval.
10. Implement Implementation and Git Automation Agent with H4 approval.
11. Implement Testing Agent post-code validation and security with H5 approval.
12. Implement final report and traceability matrix.

8. Integration Order

```
Requirement Gathering and Analysis Agent

   -> H1 Approval

   -> Project Management and Design Agent

   -> H2 Approval

   -> Pre-Code Test Planning Agent

   -> H3 Approval

   -> Implementation and Git Automation Agent

   -> H4 Approval

   -> Testing, Validation and Security Agent

   -> H5 Approval

   -> Final Reports and Traceability Matrix
```

- Do not integrate all agents at the end. Integrate one handoff at a time.
- Each agent must store outputs in the Shared Knowledge Base using the agreed schema.
- Every approval gate must update workflow state and create a workflow event.
- Every generated artifact must include project_id and traceability IDs.

9. Shared Naming Rules

Item Type	ID Format	Example
Requirement	REQ-###	REQ-001
User Story	US-###	US-001
Acceptance Criteria	AC-###	AC-001
Design Artifact	DES-###	DES-001
API Endpoint	API-###	API-001
Database Entity	DB-###	DB-001
Task / Jira Item	TASK-###	TASK-001
Test Case	TEST-###	TEST-001
Code Module	CODE-###	CODE-001
Bug Report	BUG-###	BUG-001
Security Finding	SEC-###	SEC-001
Traceability Link	TRACE-###	TRACE-001

10. Common Workflow Status Values

Status	Meaning
PENDING	Work has not started.
IN_PROGRESS	Agent or member is currently processing this stage.
WAITING_FOR_APPROVAL	Generated output is ready for human review.
APPROVED	Human reviewer approved the output.
REJECTED	Human reviewer rejected the output.
REQUEST_CHANGES	Human reviewer requested corrections.
FAILED	Execution, validation, or security check failed.
COMPLETED	Stage completed and stored successfully.

11. Pull Request Rules

- Every feature must be merged through a pull request into develop.
- The PR title must start with the component name, such as [Requirement Agent] or [Orchestrator].
- The PR description must list changed files, APIs added/updated, schema changes, test steps, and known issues.
- A PR must not include unrelated formatting or files from another member component.
- At least one other member must review the PR before merge.
- The group leader must review PRs that affect shared schemas, orchestrator logic, routing, layouts, or API contracts.

```
PR title format:

[Component Name] Short description



Example:

[Requirement Agent] Add requirement approval API



Commit message format:

component-name: short description



Examples:

requirements-agent: add requirement output schema

design-agent: add backlog generation screen

implementation-agent: add build log panel

testing-agent: add pre-code test mapping
```

```
orchestrator: add H1 to H2 transition logic
```

12. Daily Git Workflow

```
Before starting work:

git checkout develop

git pull origin develop

git checkout feature/your-branch

git merge develop



Before committing:

- Run frontend

- Run backend

- Check browser console

- Check API response

- Review changed files



Before pull request:

- Push your branch

- Complete PR checklist

- Ask reviewer to test the feature
```

13. AI Agent Usage Rules

Rules for using ChatGPT, Codex, Antigravity or any AI coding agent

- Always tell the AI the exact component, branch, allowed folders, and do-not-change files.
- Do not allow AI to rename shared schemas, routes, or status values without approval.
- Do not allow AI to create duplicate services, duplicate models, or duplicate API routes.
- After AI changes, the member must manually review changed files before committing.
- The AI output must follow the API contract and data schema documents.
- Every AI-generated feature must be tested before opening a pull request.

```
AI prompt template:
```

```
You are working on AgileFlow AI.



Component: [component name]

Branch: [branch name]

Allowed folders: [list]

Do not modify: [list]

Follow: README.md, API_CONTRACT.md, DATA_SCHEMAS.md, AI_AGENT_RULES.md



Task:

[write the exact task]



Rules:

- Use existing shared schemas.

- Do not rename existing APIs.

- Do not create duplicate models or services.

- Do not change unrelated files.

- Return a summary of changed files and testing steps.
```

14. Conflict Prevention Rules

13. Each member works only inside their assigned component folder.
14. Shared files are changed only by the group leader or after team approval.
15. API names and schema fields must match the official API_CONTRACT and DATA_SCHEMAS documents.
16. All branches must frequently merge the latest develop branch to avoid large conflicts.
17. UI design changes must use shared components instead of creating new duplicate components.
18. Incomplete work stays in feature branches. The develop branch must always run.
19. Do not push generated temporary files, environment files, secrets, node_modules, venv, or logs unless required as sample evidence.

15. Missing Function Prevention Checklist

Area	Checklist
Backend	Endpoint exists; request schema exists; response schema exists; service function exists; database function exists; error handling exists; workflow event is recorded.
Frontend	Page exists; API call exists; loading state exists; error state exists; empty state exists; status badge exists; approval buttons exist; navigation link exists.

Area	Checklist
Database	Collection exists; project_id is stored; IDs follow common format; created_at and updated_at are stored; status is valid.
Traceability	Requirement ID is connected to design, test, code, bug/security result where applicable.
Documentation	API_CONTRACT, DATA_SCHEMAS, and TASK_ASSIGNMENT are updated when shared behavior changes.

16. First Sprint Task Plan

Role	First Sprint Tasks
Group Leader / Foundation Team	Create repository; setup branches; create docs; setup frontend/backend; create database connection; create project model; create workflow and approval models; create base dashboard.
UI/Figma Design Team	Design dashboard; create project page; workflow page; agent page layout; approval gate component; traceability table; dark purple design system.
Requirement Agent Owner	Prepare requirement input screen and sample requirement JSON; implement H1 approval UI/API after foundation is ready.
Design Agent Owner	Prepare backlog/design screen and sample design output; implement H2 approval UI/API after Agent 1 output is available.
Implementation Agent Owner	Prepare generated project structure view, Git/log view, and H4 approval flow.
Testing Agent Owner	Prepare pre-code test screen, validation/security dashboard, bug/security report structure, and H3/H5 approval flow.

17. Final Team Rule

Team rule to remember

- Incomplete feature branch is acceptable.
- Broken develop branch is not acceptable.
- Every member must protect the shared workflow, schemas, and traceability model.

Appendix A - Quick Review Checklist

Before Merge Question	Yes / No
Does this PR affect only the assigned component?	
Does the code follow the agreed folder structure?	
Are API endpoints documented or unchanged?	
Are schema fields documented or unchanged?	
Does frontend run without console errors?	
Does backend start successfully?	
Are approval gates handled correctly?	
Are workflow events stored?	
Are traceability IDs preserved?	
Has another member reviewed the PR?	